

Written Analysis

Question 1:

a. Steps Cost

```
count = 0
length = len(state)
for each position i from 0 to length:
    if state[i] != goal[i]:
        count++
return count
```

This heuristic function creates a lower bound because the max count cannot be more than the displacement of letters from the initial state to the goal word. This is because each step has the same cost, meaning the max is 4 with a lower bound of 0 if they are the same word.

b. Scrabble Cost (SCRABBLE_VALUES refers to the index in the homework)

```
count = 0
length = len(state)
for each position i from 0 to length:
    if state[i] != goal[i]:
        tarChar = goal[i]
        count = count + SCRABBLE_VALUES[tarChar]
return count
```

Similarly to the steps cost, this heuristic function accounts for the total length of the word. However, it must overwrite the letter at the scrabble value cost. This means that every check can only add additional scrabble cost, creating the lower bound.

c. Frequency Cost Measure

```
count = 0
length = len(state)
for each position i from 0 to length:
    if state[i] != goal[i]:
        count++
return count
```

The rarity of the word in the dictionary is more random than having the set step counts. This means that for the frequency heuristic the lower bound underestimated more heavily by returning the number of mismatches. This is because every mismatch is at least $1 + \text{rarity}(w)$. This makes it safe to use the number of mismatches as the lower bound.

Question 2:

Word Pair	Cost Measure	UCS Cost	UCS Nodes Expanded	A* Cost	A* Nodes Expanded	Reduction
Why Ask	Steps	5	773	5	16	97.93%
Why Ask	Scrabble	9	765	9	13	98.3%
Why Ask	Frequency	38.8554	808	38.8554	755	6.56%
Bear Duck	Steps	4	691	4	6	99.13%
Bear Duck	Scrabble	11	2981	11	6	99.8%
Bear Duck	Frequency	38.2402	1084	38.2402	720	33.58%

- UCS and A* search both returned the optimal cost. Both algorithms will find the optimal cost, but UCS does not use heuristics the same way to estimate costs as A* search does.
- A* search achieved the largest percent reduction for scrabble. Step is close behind but because the scrabble values add more cost to different letters, this gives the A* search a greater advantage to sum up the scrabble cost creating less nodes expanded.
- The heuristic was least informative for frequency across both word pairs.
- Frequency does not use rarity of the word meaning the heuristics for it were heavily underestimated.

Question 3:

- Step Cost

The optimal step cost is equal when reversed because every step has the same cost. This means no matter what the words, the cost going from W_1 to W_2 is the same as going from W_2 to W_1 .

- Scrabble Cost

The optimal scrabble cost from W_1 to W_2 is not the same as going from W_2 to W_1 . This is because the cost is determined by the letter that is changed. This can be shown between the word's "dog" and "cat". From dog to cat and vice versa you get the strings "dog" and "cog". Replacing the 'd' in dog with a 'c' has a scrabble cost of 3 while replacing 'c' with a 'd' has a scrabble cost of 2. This means that every transformation is not reversible, therefore the optimal cost is also not reversible.

c. Frequency Cost

The optimal frequency cost is not reversible. This is like scrabble because the rarity from one word to another is not the same as the reverse order. This can be seen with "dog" and "cat" as well. The path the algorithm will take will go from "dog" to "don" to "dan" to "can" to "cat". Since frequency excludes the original word, the reverse will be the absolute difference of the rarity of the start and end word (W_1 and W_2).

Question 4:

a. Prompted ChatGPT-5.6 Luna with the proposed heuristic:

A natural rarity-aware heuristic is to add a **guaranteed minimum rarity cost** for the words that must still be entered.

Let:

- $d(n)$ = Hamming distance between the current word n and the goal g , i.e. $h_0(n)$.
- Each action changes exactly one letter.
- Entering a word w costs $1 + \text{rarity}(w)$.

Proposed heuristic

Define

$$h_R(n) = d(n) + \min_{\substack{w \text{ differs from } g \\ w \neq g}} \text{rarity}(w)$$

for $d(n) > 0$, with

$$h_R(g) = 0.$$

b. The AI's proposed heuristics was in the form of a formula however it gave a second version.

For a particularly clean admissible heuristic, define

$$r_{\min} = \min_{w \neq g} \text{rarity}(w)$$

and use

$$h_R(n) = \begin{cases} 0, & n = g \\ d(n)(1 + r_{\min}), & n \neq g. \end{cases}$$

- c. The proposed heuristic is a valid approach because it uses the number of mismatches as a lower bound. This is the same used for my example because the rarity is unknown meaning to not overestimate, we use the minimum possible cost which is 1.
- d. The solution cost and nodes expanded are the same because the heuristic the AI proposed was the same as the one I used. This shows that, like in Question 2, the heuristic does help manage the nodes expanded but not nearly as much as with the other costs.